ICT261 Java Lab | Unit 4

Exception Handling and Error Management

Name: Monica Nankamba Student No: 202503651

Activity A: Predict and explain

Statement	Exception	Type	Prevention / handling
Integer.parseInt("ABC");	NumberFormatException	Unchecked	Wrap in try/catch, or validate the text before parsing.
String name = null; name.length();	NullPointerException	Unchecked	Check name != null before using it.
new java.io.FileReader("missing.txt");	FileNotFoundException	Checked	Handle with try/catch or declare throws; check the file exists first.

Why OutOfMemoryError should not normally be caught to continue execution: it is an Error, not an Exception. It means the JVM has run out of memory, so the program state is unreliable. Catching it does not free memory and the program is likely to fail again immediately; the memory problem should be fixed instead.

Activity B and C: Skip or stop decision

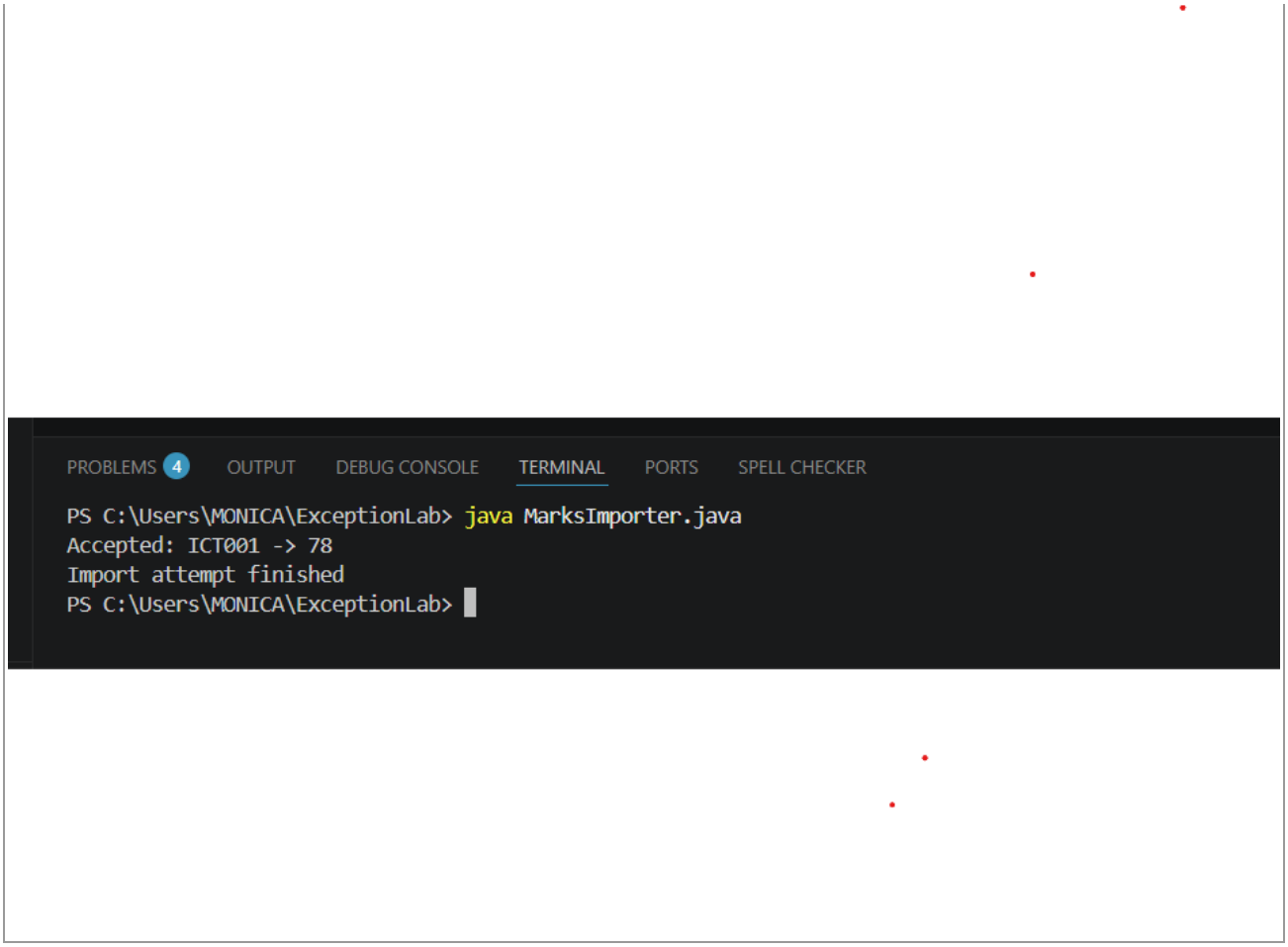
I chose to skip an invalid record and continue with the rest of the file. One bad line, such as a non-numeric mark, an out-of-range mark or a duplicate student number, should not block the valid records after it. Each problem is logged with its line number and exception type, and a friendly message is shown to the user, so nothing is lost silently. The import only stops completely when the file itself cannot be opened or read, because then no records can be processed.

Activity C: Test results

Test	Change / input	Expected behavior	Done
T1	marks.txt contains ICT001,78	Valid mark accepted	[x]
T2	ICT002,ABC	Invalid numeric value reported safely	[x]
T3	ICT001,65 after ICT001,78	Duplicate student ID identified	[x]
T4	ICT004,120	Out-of-range mark rejected	[x]
T5	Change filename to absent.txt	Friendly file-not-found message; finally runs	[x]

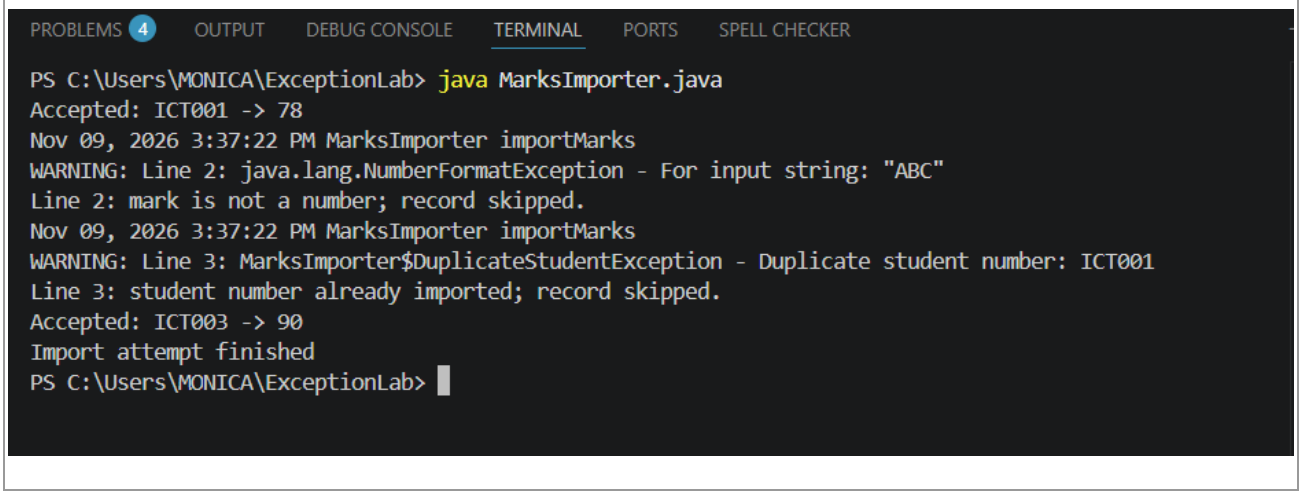
T1: marks.txt contains ICT001,78

[Paste screenshot here]



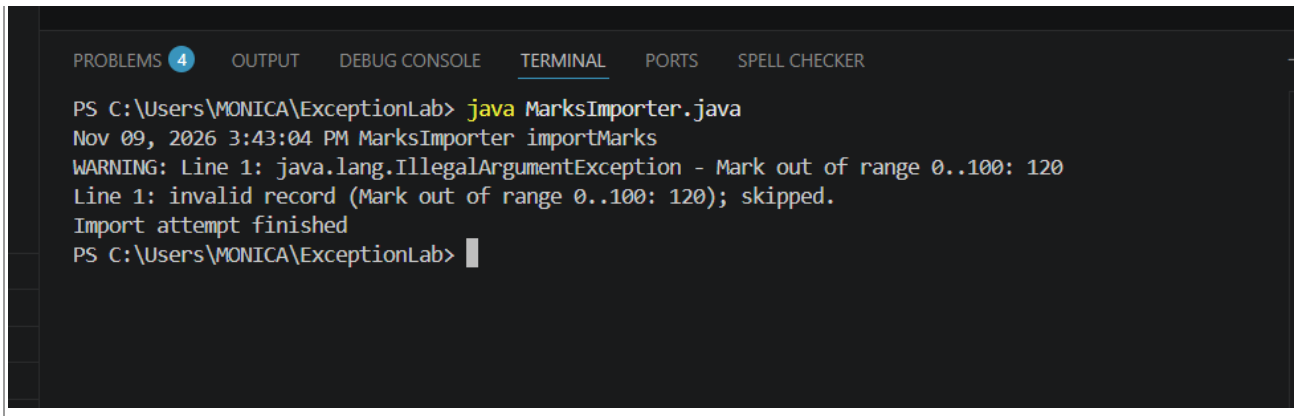
T2 and T3: original marks.txt (ICT002,ABC and duplicate ICT001)

[Paste screenshot here]



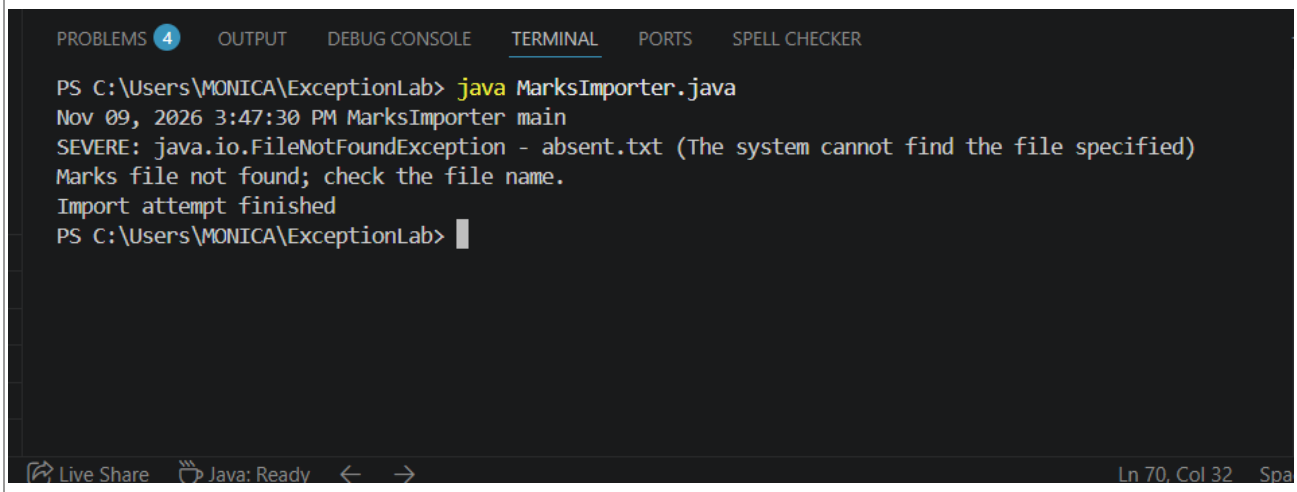
T4: marks.txt contains ICT004,120

[Paste screenshot here]



T5: filename changed to absent.txt

[Paste screenshot here]



Activity D: Engineer's review

1. Which of your exceptions is checked, and where does throws appear? Why?

IOException and FileNotFoundException are checked exceptions. throws IOException appears on the importMarks method, because opening and reading a file can fail for reasons outside my control, and Java forces me to either handle the exception or declare it. DuplicateStudentException is unchecked because it extends RuntimeException.

2. Difference between throw and throws

throw actually raises an exception object at that point in the code: throw new DuplicateStudentException(id);

throws is part of a method declaration and says the method may pass an exception to its caller: static void importMarks(String filename) throws IOException

3. Why a friendly message for the user, but technical details in the logger?

Users need a short, understandable message and do not need stack traces or class names, which would confuse them or reveal internal details. The logger keeps the exception type and cause so developers can diagnose and fix the problem later.

4. What goes wrong with an empty catch block, or catching Exception before FileNotFoundException?

An empty catch block hides the error: the program carries on with no message and no log, so bugs become very hard to find. Catching Exception before FileNotFoundException makes the specific catch unreachable, and the compiler reports an error. The specific, friendly handling would never run.